



AXON.

Agentic eXecution & Oversight Network

SYSTEM DOCUMENTATION & TECHNICAL MANUAL

Enterprise Multi-Agent Control Tower, Telemetry Observability & Governance Platform

Platform Version: 1.0.0

Release Date: July 2026

Security Class: Enterprise Confined

Developer Group: Hackathon Development Team

Table of Contents

This technical manual details the setup, operation, database layouts, and governance structures of the AXON. system:

1. Executive Summary	3
2. Architecture & Techstack	3
3. Database Structure & Schema	4
3.1 Table: agents (Agent Definitions)	4
3.2 Table: tasks (Execution Threads)	4
3.3 Table: logs (Granular Agent Outputs)	4
3.4 Table: costs (Token Spends)	5
3.5 Table: approvals (Governance Requests)	5
3.6 Table: alerts (Intercepted Anomalies)	5
4. Operations & Governance Guidelines	6
5. Installation & Execution Guide	6
6. Verification Checklists	6

1. Executive Summary

Telemetry Stream As organizations increasingly adopt multi-agent AI ecosystems, managing their execution paths, budgets, API token costs, and security compliance becomes a major challenge. AXON. (Agentic eXecution & Oversight Network) is an enterprise-grade AI Agent Control Tower serving as the central nervous system for multi-agent fleets. It addresses key operational challenges including runaway billing loops, lack of coordination, and compliance risks of fully autonomous actions by offering:	SSE (Server-Sent Events) Relational (Postgres) and NoSQL (MongoDB) databases Acyclic Graph (DAG) Lineage Map and Step Replay Logs ORM	Establishes a persistent uni-directional stream to push logs, task start/stop status, and alerts from backend to client. serving as the central nervous system for multi-agent fleets, managing their execution paths, budgets, API token costs, approvals, dependency configurations, and agent structures. transaction routing, and background thread execution.
Real-time Telemetry & Observability: Visualizes agent task execution chains through a live Directed Acyclic Graph (DAG) Lineage Map and Step Replay Logs. Cost Attribution & Guardrails: Tracks LLM token spends per agent role and enforces daily spending limits.		
Anomaly Interception: Automatically detects infinite recursion loops, halting execution to prevent token drain.	Google Gemini Pro/Flash API	Powers the real-time AI assistant (RELAY) and drives mock specialist simulations.
Human-in-the-Loop Governance: Integrates an approval queue to stage high-risk operations until approved by an operator.		

2. Architecture & Techstack

AXON. is designed with a decoupled client-server architecture utilizing robust, modern frameworks to ensure real-time responsiveness and low latency:

Tier	Technology	Purpose
Frontend	React 18, Vite, Material UI (MUI)	High-performance client UI, responsive grid modules, dark/light theme options, and custom CSS styling.
Data Visualization	Recharts, Canvas-based DAG Graph	Renders live token spend graphs, historical budgets, and real-time communication flows between agents.

3. Database Structure & Schema

agent_id VARCHAR Foreign Key (agents.id) Attributes the execution to a specific agent.

AXON. utilizes an SQLite database (located at backend/control_tower.db) with structured relations to maintain absolute referential integrity. Below is the relational mapping of each database entity:

status	VARCHAR	Not Null	Current run state: 'RUNNING', 'SUCCESS', 'FAILED', or 'PENDING_APPROVAL'.
--------	---------	----------	---

3.1 Table: agents

Stores definition profiles, models, and spending constraints for each agent type.

description	VARCHAR	-	User prompt or sub-task objective description.
-------------	---------	---	--

Column	Type	Constraints	Description
created_at	DATETIME	Default utctnow	Timestamp when the run was initiated.
id	DATETIME	Primary Key	Unique ID of the agent (e.g. 'finance_agent', 'devops_agent').

name	VARCHAR	Not Null	Descriptive name of the agent.
------	---------	----------	--------------------------------

3.3 Table: logs

Logs granular events generated by the agents during task execution.

role	VARCHAR	Not Null	Instructions defining what the agent does.
------	---------	----------	--

Column	Type	Constraints	Description
id	INTEGER	Primary Key (AI)	'IDLE'. Unique log identifier.
cost_limit	FLOAT	Not Null	Spending limit (USD) before automatic lock is triggered.
task_id	VARCHAR	Foreign Key (tasks.id)	Links the log statement to its parent execution thread.
configuration	VARCHAR	Not Null	JSON parameters for the LLM (e.g. model, temperature).
level	VARCHAR	Not Null	Severity level: 'INFO', 'WARNING', or 'ERROR'.

message	VARCHAR	Not Null	The text log details.
---------	---------	----------	-----------------------

3.2 Table: tasks

Logs all orchestrator queries and specialist sub-task runs.

timestamp	DATETIME	Default utctnow	Granular event timestamp.
-----------	----------	-----------------	---------------------------

Column	Type	Constraints	Description
id	VARCHAR	Primary Key	Unique UUID for the task execution.

3.4 Table: costs

Tracks financial expenditures and API token consumption metrics per execution.		Default PENDING	Governance status: 'PENDING', 'APPROVED', or 'REJECTED'.
status	VARCHAR		
Column	Type	Constraints	Description
reason	VARCHAR	-	Operator comments justifying the approval decision.
id	INTEGER	Primary Key (AI)	Unique cost entry ID.
reviewer	VARCHAR	-	The operator user ID executing the review (e.g. 'Admin').
task_id	VARCHAR	Foreign Key (tasks.id)	Links token usage to the specific task execution.
created_at	DATETIME	Default utcnow	Timestamp when the request entered the queue.
prompt_tokens	INTEGER	Not Null	Input tokens consumed.
updated_at	DATETIME	-	Timestamp when the decision was finalized.
completion_tokens	INTEGER	Not Null	Output tokens generated.

3.6 Table: alerts

cost_usd	FLOAT	Not Null	Dollar spend (USD) calculated from model output.
Stores anomalies (loops, budget breaches) caught by the monitoring thread.			

3.5 Table: approvals

id	INTEGER	Primary Key (AI)	Unique alert ID.
Holds governance tasks waiting for human intervention.			
task_id	VARCHAR	Foreign Key (tasks.id)	Links the alert to the offending task thread.
Column	Type	Constraints	Description
type	VARCHAR	Not Null	Type of anomaly: 'LOOP_DETECTION' or 'COST_LIMIT'.
id	VARCHAR	Primary Key	Unique approval request ID.
severity	VARCHAR	Not Null	Severity rating: 'CRITICAL' or 'WARNING'.
task_id	VARCHAR	Foreign Key (tasks.id)	Links the approval request to the suspended task.
description	VARCHAR	Not Null	Explanation of the anomaly.
action_name	VARCHAR	Not Null	The action name needing approval (e.g. 'Server Provisioning').
resolved	BOOLEAN	Default False	Indicates if the administrator has cleared the alert.

timestamp	DATETIME	Default utcnow	Timestamp of the event.

4. Operations & Governance Guidelines

```
cd backend pip install -r requirements.txt python seed_data.py python main.py
```

4.1 Infinite Loop Breaker

Note: The backend server starts by default on `http://127.0.0.1:8000`. The database will automatically initialize and populate with the default agents.

When an agent is executing a sub-task, the system monitors log consistency. If an agent emits identical warning messages or queries recursively (more than 3 times), AXON. instantly halts the task, marks the execution status as **FAILED**, registers a critical warning log, and files a `LOOP_DETECTION` alert. This shields the company from token leakage during service outages.

5.3 Frontend Installation

Navigate to the frontend directory, install npm packages, and spin up the Vite development server:

4.2 Human in the Loop Approval Center

```
cd frontend npm install npm run dev
```

High-risk transactions or tasks requesting cost resource margins above budget limits trigger an automatic hold state. The task's execution freezes, its state shifts to `PENDING` (for the next available node).

Note: The frontend development server will launch on `http://localhost:5173`, (or the next available port).

Open this URL in your web browser to explore AXON.

The approval is distributed. Developers or administrators review the context and click "Approve" or "Reject". On approval, the task is unlocked and completes successfully.

6. Verification Checklists

4.3 System Settings Console

Unlocked via logs (status `are verified during demo`). In the top right Settings Drawer, this console allows operators to configure execution speed delays, loop breaker caps, and cost caps globally. Additionally, administrators can perform a hard database refresh, purging telemetry history and reinstating default seed agents with a single click.

- Engine Runs:** Input prompts in the console and watch the logs stream in.
- Stop Button:** Click "Stop Engine" during execution to verify instant thread cancellation.

5. Installation & Execution Guide

Follow the "Final Approval Request" scenario, then approve the pending item inside the approvals center.

4. Loop Anomaly: Run the "Retry Loop Anomaly" scenario and watch the alert feed register the loop breach.

5.1 Prerequisites

- Python 3.12+ installed
- Node.js (v18+) and npm installed
- Git CLI tools

5.2 Backend Installation

Navigate to the backend directory, install requirements, and run the FastAPI server: